WIFI Network Basics

---

1. Electromagnetic Waves and Wireless Communication

1.1 What Are Electromagnetic Waves

The essence of WiFi is **electromagnetic waves** — a changing electric field generates a magnetic field, and a changing magnetic field generates an electric field, and the two alternately excite each other and propagate through space.

Key Parameter	Description
Frequency (f)	Number of oscillations per second, in Hz. WiFi uses 2.4 GHz = 2.4 billion oscillations per second
Wavelength (λ)	Physical length of one complete waveform. λ = c / f ≈ 0.125 m (about 12.5 cm at 2.4 GHz)
Amplitude	Signal strength, affected by distance and obstacles
Modulation	The way data is "encoded" onto a carrier wave (AM/FM/PM)

> The higher the electromagnetic wave frequency, the shorter the wavelength, the weaker the penetration capability (greater wall penetration loss), but the larger the amount of data it can carry.

1.2 Why WiFi Uses 2.4 GHz

Band	Wavelength	Characteristics
2.4 GHz	~12.5 cm	Strong wall penetration, but many devices and heavy interference (microwave ovens and Bluetooth also share this band)
5 GHz	~6 cm	Fast, less interference, but weak wall penetration
6 GHz (WiFi 6E)	~5 cm	Newest band, many channels, but shortest range

The ESP32-S3 **only supports 2.4 GHz**. A 5G WiFi router must have the 2.4 GHz band enabled at the same time for the ESP32-S3 to connect.

---

2. How WiFi Works

2.1 Channels

The 2.4 GHz band is divided into **14 channels** (China can use 1~13), each channel spaced 5 MHz apart, but the WiFi signal itself occupies 22 MHz of bandwidth.

```
Channel:   1        6        11
Frequency: 2412MHz  2437MHz  2462MHz
        ████             ████        <- Non-overlapping (1/6/11 recommended)
```

> **Channel selection principle**: Adjacent routers should select different channels among 1, 6, and 11 to avoid co-channel interference.

2.2 RSSI (Signal Strength)

RSSI (Received Signal Strength Indicator) indicates how strong the signal "heard" by the receiving end is:

RSSI Range	Signal Quality	Real-world Experience
> -50 dBm	Excellent	Router right next to you
-50 ~ -65 dBm	Good	Normal use
-65 ~ -75 dBm	Fair	Possible occasional lag
-75 ~ -85 dBm	Weak	Edge area, prone to disconnection
< -85 dBm	Very weak	Basically unusable

> dBm is a logarithmic unit: every 3 dB decrease halves the signal power. `-50 dBm` is 1000 times stronger than `-80 dBm`.

2.3 Evolution of the 802.11 Protocol Family

Standard	Year	Band	Max Rate	Common Name
802.11b	1999	2.4 GHz	11 Mbps	—
802.11g	2003	2.4 GHz	54 Mbps	—
802.11n	2009	2.4/5 GHz	150 Mbps	WiFi 4
802.11ac	2013	5 GHz	866 Mbps	WiFi 5
802.11ax	2019	2.4/5/6 GHz	1200 Mbps	WiFi 6

The ESP32-S3 supports **802.11 b/g/n**, with a theoretical maximum rate of 150 Mbps (in HT40 mode) and an actual throughput of about 20~50 Mbps.

2.4 WiFi Security Authentication

Authentication Mode	Encryption Algorithm	Security	ESP32-S3 Support
OPEN	None	Insecure	☑
WEP	RC4	Obsolete (crackable in minutes)	☑
WPA-PSK	TKIP	Weak	☑
WPA2-PSK	**AES-CCMP**	**Secure (recommended)**	☑
WPA3-PSK	SAE	More secure	☑
WPA2-Enterprise	AES + RADIUS	Enterprise level	☑

---

3. WiFi Operating Modes

3.1 STA Mode (Station)

The ESP32 connects to an existing router like a phone does, obtains an IP address, and then accesses the network.

```
Internet <-> [Router/AP] <-> WiFi <-> [ESP32-STA]
                                IP: allocated by DHCP (e.g. 192.168.1.100)
```

Characteristics:

- IP is assigned by the router's DHCP (not fixed unless static IP is set)
- Can access the Internet and the local network
- The foundation of all subsequent network examples (UDP/TCP/HTTP/MQTT)

3.2 AP Mode (Access Point)

The ESP32 **becomes a hotspot itself**, and other devices connect to it.

```
                [ESP32-AP]
                IP: 192.168.4.1 (fixed)
                DHCP Server: assigns IPs to clients
               /    |    \
         [Phone]  [PC]  [Another ESP32]
           .2      .3        .4
```

Characteristics:

- IP is fixed at 192.168.4.1, and the ESP32 acts as its own DHCP server
- Cannot access the Internet (unless NAT forwarding is done)
- Used for provisioning, direct connection control, and offline LAN communication

3.3 STA + AP Hybrid Mode

The ESP32 acts as both a Station and an Access Point:

```
Internet <-> [Router] <-> [ESP32] <-> [Phone connects to ESP32 hotspot]
                WiFi STA      WiFi AP
```

Typical application: **SmartConfig provisioning** — the phone connects to the ESP32 hotspot, tells it the router password, then the ESP32 switches to STA mode to connect to the router.

3.4 STA vs AP Comparison

Comparison	STA Mode	AP Mode
Role	Client (connects to others)	Hotspot (connected by others)
IP source	Obtained dynamically via DHCP	Fixed 192.168.4.1
DHCP service	Provided by router	Provided by ESP32 itself
Max connections	N/A	Configurable (default 4, max 10)
Network access	Can access the Internet	LAN only
Core API	`esp_wifi_connect()`	`esp_wifi_start()` (no connect needed)
Typical scenario	IoT devices going online, data reporting	Provisioning, direct connection control

---

4. ESP32-S3 WiFi Hardware Features

Feature	Parameter
WiFi standard	IEEE 802.11 b/g/n
Operating band	2.4 GHz
Max transmit power	+21 dBm (11b)
Max receive sensitivity	-97 dBm (11b)
Supported protocols	WPA/WPA2/WPA3 personal/enterprise
Antenna	On-board ceramic antenna / IPEX external antenna
Operating modes	STA / AP / STA+AP / Sniffer
Channel range	1 ~ 14

5. ESP32-S3 WiFi Development Essentials

5.1 Resource Conflict Between WiFi and ADC2

The ESP32-S3 has two ADC units: ADC1 and ADC2. **ADC2 shares internal hardware resources with WiFi**:

Scenario	ADC1	ADC2
WiFi off	Normal	Normal
WiFi on	**Normal**	**Readings abnormal / unavailable**

> In development practice, for any project that enables WiFi, analog acquisition must use ADC1 (GPIO 1~9 channel range).

5.2 NVS — WiFi's Storage Dependency

The WiFi driver needs to read the RF calibration parameters stored in NVS (Non-Volatile Storage) during initialization. **Whether or not WiFi configuration is saved, `nvs_flash_init()` is required.**

The WiFi driver depends on the RF calibration parameters stored in NVS; the NVS partition must be initialized before startup — if the partition is full or the version is incompatible, erase it and reinitialize.

```
// Initialize the NVS flash; the WiFi driver depends on its RF calibration
parameters
esp_err_t ret = nvs_flash_init();
if (ret == ESP_ERR_NVS_NO_FREE_PAGES || ret == ESP_ERR_NVS_NEW_VERSION_FOUND) {
    // NVS partition full or version mismatch — erase and retry
    ESP_ERROR_CHECK(nvs_flash_erase());
    ret = nvs_flash_init();
}
```

5.3 Fixed WiFi Initialization Order

The ESP-IDF WiFi initialization order is **strictly non-interchangeable**; missing a step causes a crash:

WiFi initialization has strict order requirements; all ten steps below are indispensable, and swapping the order will cause startup failure.

```
1. nvs_flash_init()           <- NVS storage
2. esp_netif_init()           <- TCP/IP network stack initialization
3. esp_event_loop_create_default() <- event loop
4. esp_netif_create_default_wifi_sta()  (or _ap())  <- network interface
5. esp_wifi_init(&cfg)        <- WiFi driver
6. Register event callbacks          <- WIFI_EVENT + IP_EVENT
7. esp_wifi_set_mode()        <- STA / AP / APSTA
8. esp_wifi_set_config()      <- SSID + Password
9. esp_wifi_start()           <- start WiFi
10. esp_wifi_connect()         <- initiate connection in STA mode (not needed in
AP mode)
```

> If `esp_wifi_connect()` is called immediately after `esp_wifi_start()`, it may crash; it is recommended to add `vTaskDelay(500ms)` to wait for the underlying layer to be ready.

5.4 Antenna

ESP32-S3 development boards usually use an **on-board ceramic antenna** (the small gray block on the PCB). Some core boards provide an **IPEX connector** for connecting an external high-gain antenna:

Antenna Type	Gain	Suitable Scenario
On-board ceramic antenna	~2 dBi	Short range, general use
IPEX external rod antenna	~5 dBi	Wall penetration, long range

> When using an IPEX external antenna, you need to configure the antenna selection GPIO in menuconfig or by calling `esp_wifi_set_ant_gpio()` in code.

5.5 Strapping Pins

The ESP32-S3's **GPIO0, GPIO3, GPIO45, and GPIO46** are strapping pins (they determine the chip's operating mode at startup) and have internal pull-up/pull-down resistors. **These pins cannot be used for ADC input**, because the internal resistors raise/lower the level so that the reading always stays near VCC or GND.

5.6 WiFi Event-Driven Model

ESP-IDF's WiFi uses an **event-driven** architecture. WiFi state changes generate events at the underlying layer, and the developer registers callback functions to handle them:

When the underlying WiFi state changes, the corresponding event is triggered, and the callback function dispatches to different handling branches based on the event type.

```
WiFi state change -> WIFI_EVENT / IP_EVENT -> event_handler()
                                    |- WIFI_EVENT_STA_START       -> auto
connect
                                    |- WIFI_EVENT_STA_DISCONNECTED -> clear flag
+ reconnect
                                    |- WIFI_EVENT_SCAN_DONE        -> print AP
list
                                    |- WIFI_EVENT_AP_START         -> hotspot
created
                                    |- WIFI_EVENT_AP_STACONNECTED  -> a device
connected
                                    |- WIFI_EVENT_AP_STADISCONNECTED -> a device
disconnected
                                    `- IP_EVENT_STA_GOT_IP       -> set flag +
print IP
```

5.7 EventGroup Synchronization Mechanism

WiFi connection is **asynchronous** — `esp_wifi_connect()` returns immediately after being called, and the actual connection result is notified by an event callback. Use the **FreeRTOS event group** to achieve synchronous waiting:

WiFi connection is an asynchronous operation; the main thread blocks and waits for the connection result through an event group, and the callback sets a bit to notify and unblock it.

```c
// Main thread: block and wait for WiFi connection success
xEventGroupWaitBits(wifi_event_group,   // event group handle
                    WIFI_CONNECTED_BIT, // event bit to wait for
                    // don't clear the bit on exit
                    pdFALSE,
                    // wait for any bit (not all)
                    pdFALSE,
                    timeout);           // blocking timeout in ticks

// After getting IP, set the bit and wake up the main thread
xEventGroupSetBits(wifi_event_group, WIFI_CONNECTED_BIT);
```

---

6. WiFi OTA Firmware Upgrade

6.1 What Is OTA

OTA (Over-The-Air) is one of the most important WiFi applications — the device downloads a new firmware via WiFi, writes it to Flash itself, and restarts to switch over, with no USB cable or JTAG programmer needed throughout.

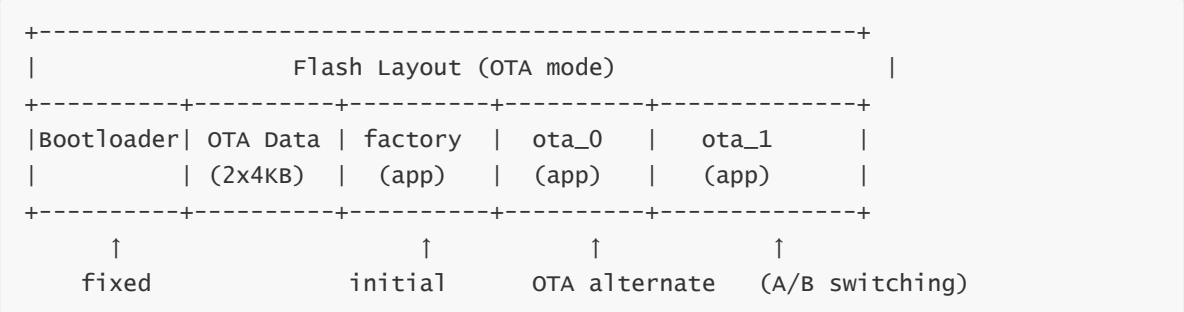
Upgrade Method	Operation	Applicable Stage	Physical Connection Needed
USB serial flashing	USB cable + idf.py flash	Development/debugging	Yes
JTAG flashing	JTAG debugger	Development/debugging	Yes
WiFi OTA	**HTTP download + partition switching**	**Mass-production products**	**No**

> OTA is the **core operation capability** of IoT products — deployed devices can be fixed and enhanced without being recalled.

6.2 Flash Partition Layout

OTA requires at least two APP partitions used alternately (A/B switching) to prevent the device from being bricked if power is lost mid-upgrade:

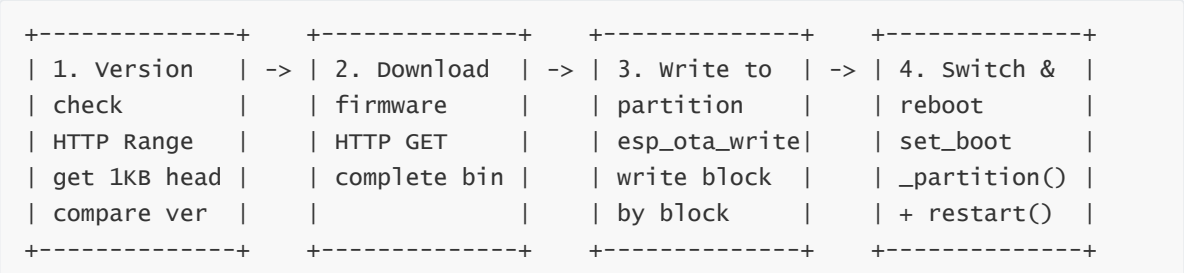
The following is the Flash partition layout in OTA mode; factory stores the factory firmware, ota_0 and ota_1 alternately receive new firmware, and OTA Data records the metadata for the next boot partition.



- **factory**: the factory firmware, written during mass-production flashing
- **ota_0 / ota_1**: two partitions used alternately. Currently running on ota_0 → OTA writes to ota_1 → restart boots from ota_1 → next OTA writes to ota_0
- **OTA Data**: records the "which partition to boot from next" metadata (2 sectors backing each other up)

> **Why two APP partitions?** If power is lost while writing new firmware, the old partition is unaffected, and the device still runs the old version normally after restart. This is OTA's "anti-bricking" mechanism.

6.3 OTA Upgrade Process



Step 1 — Version check: Use an HTTP Range request to download only the first 1024 bytes of the firmware, search for the `esp_app_desc_t` magic number (`0xABCD5432`) to locate the version string, and compare it with the local `esp_app_get_description()->version`. If the server version <= local version, skip the upgrade.

Step 2 — Download firmware: After confirming a new version exists, use HTTP GET to download the complete `.bin` firmware file.

Step 3 — Write block by block: `esp_ota_begin()` opens the target partition → loop `esp_http_client_read()` + `esp_ota_write()` to write → `esp_ota_end()` closes and verifies.

Step 4 — Switch boot: `esp_ota_set_boot_partition()` modifies OTA Data to point to the new partition → `esp_restart()` restarts.

6.4 Version Extraction — Magic Number Search Method

The layout of the `esp_app_desc_t` structure in the bin file:

The firmware header contains a magic number identifier and version fields; the version information is located by searching for a fixed byte sequence — the magic number 0xABCD5432 is at offset 0x00, and a 32-byte version string is at offset 0x10.

```
Offset      Content
------------------------------------------------------------
0x00      magic_word = 0xABCD5432          <- magic (little-endian: 32 54 CD AB)
0x04      secure_version
0x08      reserved fields
0x10      version[32]                      <- version string, e.g. "1.0.0"
...
```

Search process:

1. HTTP Range `bytes=0-1023` to download the first 1KB of the firmware header
2. Match `{0x32, 0x54, 0xCD, 0xAB}` byte by byte
3. Magic offset +0x10 → read the 32-byte version string
4. Verify that the first character is '0'~'9'

6.5 OTA Core APIs

OTA operations follow a six-step flow of "get partition -> begin write -> write block by block -> end and verify -> set boot partition -> restart and switch", with each step corresponding to a core API.

```c
// Get the "next" writable OTA partition (the one other than the currently
running partition)
const esp_partition_t *esp_ota_get_next_update_partition(NULL);

// Begin OTA write (OTA_SIZE_UNKNOWN = unknown firmware size)
esp_ota_begin(partition, OTA_SIZE_UNKNOWN, &handle);

// Write in chunks (4KB max)
esp_ota_write(handle, data, size);

// End write (verify + close)
esp_ota_end(handle);

// Set the next boot partition (modify OTA Data metadata)
```

```
esp_ota_set_boot_partition(partition);

// Reboot to new firmware
esp_restart();
```

6.6 Firmware Server

OTA requires an HTTP file server to host the `.bin` file. During the testing phase, you can start a simple server on the PC itself:

A single command on the PC starts an HTTP server, and the ESP32 accesses this address via WiFi to download the firmware.

```
# Start HTTP server with Python 3 (port 8080, root = current directory)
python -m http.server 8080

# On Windows, ensure the firewall allows port 8080
```

On the ESP32 side, `OTA_URL` points to `http://<PC_IP>:8080/WiFi_OTA.bin`.

6.7 OTA Notes

Point	Description
The partition table must contain OTA partitions	menuconfig -> Partition Table -> "Factory app, two OTA definitions"
Firmware size limit	ota_0/ota_1 each occupy half of the Flash minus factory; exceeding it causes `esp_ota_write` to fail
Network stability	If WiFi disconnects mid-download, `esp_ota_abort()` rolls back and the device still runs the old version
Version number format	Numbers and dots only (`1.0.0`), no `v` prefix
Security	Production environments should use HTTPS + certificate verification + firmware signing to prevent man-in-the-middle attacks
Rollback	ESP-IDF supports automatic rollback — if the new firmware restarts N times consecutively, it automatically switches back to the old version

> For the complete code implementation and operating steps, see (5.Network Protocols and Wireless Communication -> 12.WIFI-OTA Upgrade -> WIFI-OTA Upgrade).

7. WiFi Common Problem Troubleshooting

Symptom	Cause	Solution
Always prints "Disconnected, reconnecting..."	Wrong SSID or password	Check the `WIFI_SSID` and `WIFI_PASS` macros
Fails to connect to 5G WiFi	ESP32-S3 only supports 2.4G	Make sure the router's 2.4G band is enabled
Got IP but cannot ping the Internet	Router has no Internet connection / DNS issue	Ping the router IP first to confirm the LAN is reachable
`esp_wifi_connect()` returns an error	WiFi not fully initialized	Ensure `vTaskDelay(500ms)` is added after `esp_wifi_start()`
`WIFI_EVENT_STA_DISCONNECTED` triggers frequently	Weak signal or congested router channel	Get closer to the router / change the WiFi channel
Compile error: `nvs_flash.h` not found	NVS not initialized	Add `nvs_flash_init()` at the beginning of `app_main()`
Phone cannot find the hotspot in AP mode	Channel occupied / not started properly	Check the log to confirm `WiFi AP started`; try changing `AP_CHANNEL`
AP mode prompts "wrong password" when connecting	Password shorter than 8 characters	WPA2 requires a password of >= 8 characters

Common Network Communication Protocols

> **Importance**: The TCP/IP protocol stack is the underlying foundation of all network communication on the ESP32-S3 — whether you use WiFi, Ethernet, HTTP, MQTT, or OTA upgrade, data is ultimately transmitted over the network through the TCP/IP protocol stack. Understanding the layered model, IP addressing, the port mechanism, the Socket API, and the core differences between TCP and UDP is a prerequisite for troubleshooting network problems, optimizing communication performance, and designing reliable IoT applications. This chapter does not involve specific code flashing, but instead establishes a **theoretical framework** for network communication — all API calls, parameter configuration, and exception troubleshooting in the subsequent network examples (UDP/TCP/HTTP/MQTT) are based on the knowledge in this chapter.

1. The TCP/IP Protocol Stack

1.1 The Layered Model

The essence of network communication is **layer-by-layer encapsulation and peer-to-peer communication**:

```
+-------------------------------------------------------------+
|               Application Layer                             |  HTTP / MQTT /
DNS / OTA
|  http://example.com/api  ->  "Hello ESP32"                  |
+-------------------------------------------------------------+
|               Transport Layer                               |  TCP / UDP
|  Port numbers (80/8080/1883) + reliable/unreliable transport  |
+-------------------------------------------------------------+
|               Internet Layer                                |  IP
|  IP address (192.168.1.100) + routing + fragmentation         |
+-------------------------------------------------------------+
|               Link Layer                                    |  WiFi /
Ethernet
|  MAC address (AA:BB:CC:DD:EE:FF) + channel access           |
+-------------------------------------------------------------+
```

1.2 Data Encapsulation Process

When sending data, each layer adds its own header, like **Russian nesting dolls**:

```
Application layer:  [ HTTP GET /index.html ]
                         | add TCP header
Transport layer:  [ TCP header | src_port=12345 dst_port=80 | HTTP GET ... ]
                         | add IP header
Internet layer:  [ IP header | src=192.168.1.100 dst=93.184.216.34 | TCP header |
HTTP GET ... ]
                         | add WiFi frame header
Link layer:  [ WiFi frame header | src=AA:BB:CC dst=DD:EE:FF | IP header | TCP
header | HTTP GET ... | FCS checksum ]
```

The receiving end de-encapsulates layer by layer until the application layer gets the original data.

1.3 Key Protocol Comparison

Protocol	Layer	Reliability	Connection	Header Overhead	Typical Use
IP	Internet layer	Unreliable	Connectionless	20B	Addressing, routing
ICMP	Internet layer	Unreliable	Connectionless	8B	Ping, error reporting
TCP	Transport layer	**Reliable**	**Connection-oriented**	20B	Web, files, MQTT
UDP	Transport layer	Unreliable	Connectionless	8B	DNS, video streams, sensors
ARP	Link layer	—	—	28B	IP->MAC address resolution
DHCP	Application layer	—	—	—	Automatic IP acquisition

---

2. IP Address

The IP address is the "door number" of network communication — the ESP32 can only be accessed by other devices on the LAN after obtaining an IP, and it is a prerequisite for all subsequent Socket communication, HTTP requests, and MQTT connections.

2.1 IPv4 Basics

An IPv4 address consists of **32 bits**, written as 4 decimal segments (each 0~255):

```
192     .   168     .   1       .   100
11000000  10101000  00000001  01100100     <- 32-bit binary
+-----------------------+  +-------+
           network part            host part
      (the subnet mask determines the boundary)
```

2.2 Subnet Mask

The subnet mask determines which part of an IP address is the "network number" and which is the "host number":

Subnet Mask	CIDR	Usable IPs	Common Scenario
255.255.255.0	/24	254	Home router
255.255.0.0	/16	65534	Large enterprise

```
IP:      192.168.  1.100
Mask:   255.255.255.  0
        ==========   ===
         network     host part (0~255, where 0=network number, 255=broadcast)
```

> All devices in the same network have the same "network part" and different "host parts".

2.3 Public IP vs Private IP

Range	Type	Description
10.0.0.0 ~ 10.255.255.255	Private Class A	Large enterprise internal
172.16.0.0 ~ 172.31.255.255	Private Class B	Medium enterprise internal
192.168.0.0 ~ 192.168.255.255	**Private Class C (home Ethernet)**	**Most common — home routers**
Others	Public IP	Unique on the Internet, must be purchased

> Private IPs cannot be directly accessed from the Internet — the router maps private IPs to public IPs through **NAT (Network Address Translation)**.

2.4 Special IP Addresses

IP	Meaning	Use
`127.0.0.1`	Local loopback (localhost)	Accessing yourself
`0.0.0.0`	Any address	Binding to all network interfaces
`255.255.255.255`	Broadcast address	Sending to all devices on the LAN
`192.168.1.1`	Usually the router gateway	Entry to the admin page

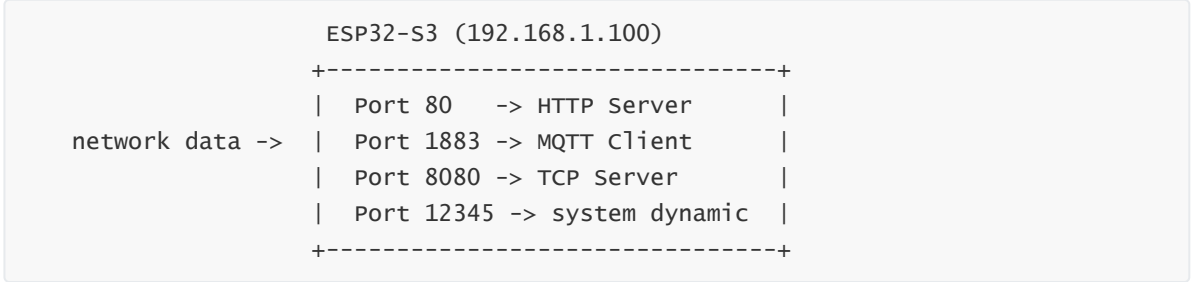
---

3. Ports

The port is the key to "multiple services on one device" — the ESP32 can simultaneously run an HTTP server (80), an MQTT client (1883), and a TCP data channel (8080), with each service bound to a different port without interfering. Firewall and NAT configuration also depends on port numbers.

3.1 The Concept of Ports

The IP address locates the device, and the port locates the program on the device.



Ports are 16-bit numbers (0~65535):

Range	Type	Description
0 ~ 1023	Well-known ports	HTTP=80, HTTPS=443, MQTT=1883, DNS=53
1024 ~ 49151	Registered ports	Custom services (e.g. 8080)
49152 ~ 65535	Dynamic ports	Temporary use by clients

3.2 Client Port vs Server Port

```
Client (browser/ESP32)                    Server
---------------------------          -----------------
socket()                                  socket()
(system auto-assigns temporary              bind(80)          <- must use a
fixed port,
 port, e.g. 52341)                                          otherwise the client
                                                              won't know which
port to connect to
connect(93.184.216.34:80) ------ >  listen(80) + accept()
                                    received connection: client IP:52341 <->
server IP:80
send() ------------------------- >  recv() -> process -> send()
recv() <--------------------------  (reply)
```

> **The server must `bind()` a fixed port**, otherwise the client won't know where to connect; the client port is auto-assigned by the OS (used and discarded).

---

4. Socket Programming Basics

Socket is the **core API layer** of ESP-IDF network programming — whether it is custom TCP/UDP communication, an HTTP client, or an MQTT client, data is sent and received through Sockets underneath. Mastering Socket data structures, byte order conversion, and creation methods is the basis for reading and writing all network code.

4.1 What Is a Socket

Socket is the network programming interface provided by the operating system — developers send and receive data through Sockets without worrying about details such as underlying WiFi frames, IP fragmentation, and TCP retransmission.

```
Application code                   OS kernel
---------                         -------------
send(sock, "Hello", 5)
    |                              +-------------+
    +------------------------------>   TCP stack  | -> segment -> add TCP header
                                   +-------------+
                                   |   IP stack   | -> fragment -> add IP header
-> route
                                   +-------------+
                                   |  WiFi driver | -> add WiFi frame -> DMA ->
antenna transmit
                                   +-------------+
```

4.2 Key Data Structures

The two most core address structures in Socket programming — `sockaddr` is the generic type and `sockaddr_in` is the IPv4-specific type; all Socket APIs use the latter via a cast.

```
// Generic socket address (IPv4/IPv6)
struct sockaddr {
    // Address family: AF_INET (IPv4) or AF_INET6 (IPv6)
    sa_family_t sa_family;
    char        sa_data[14];  // Protocol address (IP + port)
};
struct sockaddr_in {
    sa_family_t    sin_family; // Address family: IPv4
    // Port number (network byte order)
    uint16_t       sin_port;
    struct in_addr sin_addr;   // IP address
};
struct sockaddr_in server_addr;
bind(sock_fd, (struct sockaddr *)&server_addr, sizeof(server_addr));
```

4.3 Network Byte Order

Network byte order = Big-Endian, meaning the high-order byte is stored at the low address. The ESP32 CPU is little-endian, opposite to network order, so conversion is required:

```
Number 0x12345678 (32-bit)

Big-Endian (network order):
low address -> high address
  12 | 34 | 56 | 78     <- high byte first

Little-Endian (ESP32 host order):
low address -> high address
  78 | 56 | 34 | 12     <- low byte first
```

Function	Meaning	Use
`htons(n)`	Host TO Network Short (16-bit)	Port number conversion
`htonl(n)`	Host TO Network Long (32-bit)	IP address conversion (rarely used)
`ntohs(n)`	Network TO Host Short	Restore after receiving
`ntohl(n)`	Network TO Host Long	Restore after receiving
`inet_pton()`	String -> binary IP (automatic network order)	`"192.168.1.1"` -> `0x0101A8C0`

Both ports and IP addresses need to be converted to network byte order — `htons()` converts the port, and `inet_pton()` converts the IP string to network-order binary.

```
// Correct: use conversion functions
// Convert to network byte order
addr.sin_port = htons(8080);
// String -> binary (network order)
inet_pton(AF_INET, "192.168.1.100", &addr.sin_addr);

// Wrong: direct assignment
// CPU is little-endian, network sees 0x901F
addr.sin_port = 8080;
```

4.4 Socket Creation Types

`socket()` distinguishes TCP (stream) from UDP (datagram) via the `type` parameter — this is the first step of all subsequent Socket operations and determines the communication mode.

```
int fd = socket(domain, type, protocol);  // General form
int tcp_sock = socket(AF_INET, SOCK_STREAM, IPPROTO_TCP);
int udp_sock = socket(AF_INET, SOCK_DGRAM, IPPROTO_UDP);
```

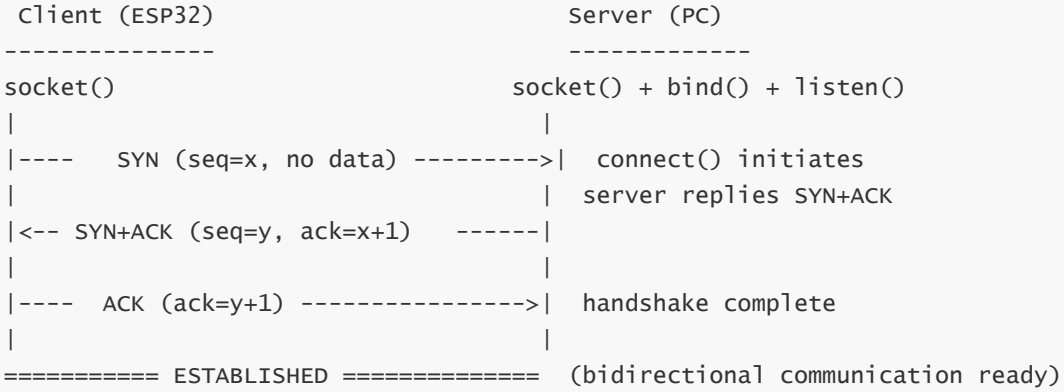
Parameter	TCP	UDP
domain	AF_INET (IPv4)	AF_INET (IPv4)
type	**SOCK_STREAM** (stream)	**SOCK_DGRAM** (datagram)
Behavior	Connection-oriented, reliable, byte stream	Connectionless, unreliable, preserves message boundaries

---

5. TCP in Depth

TCP is the most commonly used reliable transport protocol for IoT devices — firmware OTA upgrades cannot lose a single byte, MQTT messages must be delivered reliably, and HTTP pages must load completely. All these scenarios rely on TCP's acknowledgment and timeout retransmission mechanisms. Understanding the three-way handshake, four-way termination, and the `recv()` return value is the basis for running a stable TCP service.

5.1 Three-Way Handshake



- SYN: request to establish a connection
- SYN+ACK: agree + request (the server also confirms its own direction)

- ACK: confirmation (officially established)

> The essence of the three-way handshake is **that both sides confirm the other's send/receive capability is normal**.

5.2 Four-Way Termination

```
 Client                                  Server
---------                               ---------
close() or shutdown()
|---- FIN (seq=u) ------------------>|  Client: "I have no more data"
|<--- ACK (ack=u+1)  ----------------|  Server: "received"
|                                   |  (server may still have data to send)
|<--- FIN (seq=v)  ------------------|  Server: "I have no more data either"
|---- ACK (ack=v+1) ---------------->|  Client: "received"
|                                   |
======== connection closed ============
```

> TCP is full-duplex — each direction closes independently. `close()` closes both directions; `shutdown(fd, SHUT_WR)` closes only the write direction (you can still receive).

5.3 How Reliable Transmission Is Implemented

TCP reliability comes from three mechanisms:

Mechanism	Principle
Acknowledgment (ACK)	Reply with an ACK after receiving data; the sender retransmits if no ACK is received
Sequence number (SEQ)	One number per byte; the receiver sorts and reassembles by sequence number
Timeout retransmission (RTO)	Start a timer after sending; retransmit if no ACK is received before timeout

5.4 The Three Return Cases of `recv()`

Return Value	Meaning	Handling
`> 0`	Received n bytes of data	Process normally
`= 0`	**The peer closed the connection normally** (sent FIN)	`close()` and `break`
`< 0`	No data available (`MSG_DONTWAIT`) or an error	Check `errno`

The three return values of `recv()` correspond to three handling branches — this is the standard template for TCP receive logic.

```c
int ret = recv(sock_fd, buf, len, 0);
if (ret > 0)     { /* Data received, process normally */ }
else if (ret == 0) { /* Peer closed, break and close */ }
else             { /* No data (non-blocking), or real error */ }
```

5.5 Standard TCP Server Flow

```
socket() -> bind() -> listen() -> while(1) {
    client_fd = accept() -> while(1) {
        recv() / send()    <- communicate with a single client
    } -> close(client_fd)   <- client disconnected, back to accept
}
```

---

6. UDP in Depth

UDP's low overhead and low latency make it the first choice for scenarios such as high-frequency sensor data reporting, real-time audio/video streams, and DNS queries. In IoT, when "speed takes priority over reliability" — such as temperature and humidity collection hundreds of times per second, or device discovery broadcasts — UDP is more suitable than TCP.

6.1 The Meaning of Connectionless

UDP does not handshake, establish connections, or guarantee delivery — datagrams are like **throwing a message in a bottle**:

```
      Client (ESP32)                                Server (PC)
     ---------------                               -------------
     socket(SOCK_DGRAM)                          socket(SOCK_DGRAM) + bind(8080)
     |                                             |
     |-- sendto("Hello", 192.168.1.183:8080) -->|  direct send
     |                                             |  (if it arrives, it arrives; if
not, forget it)
     |<-- sendto("Reply", 192.168.1.100:52341) --|  reply
```

6.2 UDP Suitable Scenarios

☑ Suitable for UDP	❌ Not suitable for UDP
Video/audio real-time streams (occasional glitches OK)	File transfer (not a single byte can be lost)
DNS queries (small request volume, timeout-retry is fine)	Web browsing (HTML must be complete)
High-frequency sensor data (losing a few frames is fine)	Firmware OTA upgrades (firmware must be complete)
Broadcast/multicast messages	Financial transactions (packet loss = money loss)

6.3 UDP vs TCP API Comparison

Comparison of the TCP and UDP API call flows — TCP needs `listen()` + `accept()` to establish a connection and then communicate via the file descriptor; UDP directly uses `sendto()` / `recvfrom()` carrying the target address.

```
// ===== TCP Server =====                    // ===== UDP Server =====
socket(AF_INET, SOCK_STREAM, 0)              socket(AF_INET, SOCK_DGRAM, 0)
bind(fd, addr, len)                          bind(fd, addr, len)
listen(fd, 5)                                // No listen
// No accept, direct recvfrom
client_fd = accept(fd, ...)
recv(client_fd, buf, len, 0)                 recvfrom(fd, buf, len, 0, &addr,
&len)
send(client_fd, buf, len, 0)                 sendto(fd, buf, len, 0, &addr, len)

// ===== TCP Client =====                    // ===== UDP Client =====
socket(AF_INET, SOCK_STREAM, 0)              socket(AF_INET, SOCK_DGRAM, 0)
connect(fd, &addr, len)                      // No connect
send(fd, buf, len, 0)                        sendto(fd, buf, len, 0, &addr, len)
recv(fd, buf, len, 0)                        recvfrom(fd, buf, len, 0, &addr,
&len)
```

6.4 Non-Blocking Receive (MSG_DONTWAIT)

The `MSG_DONTWAIT` flag makes `recvfrom()` / `recv()` **non-blocking** — when there is no data, it immediately returns -1 (`errno=EAGAIN`) without blocking the task, which is suitable for high-frequency polling scenarios (such as a low-latency receive loop every 10 ms).

```
int recv_len = recvfrom(sock_fd, recv_buf, sizeof(recv_buf)-1,
                        MSG_DONTWAIT,       // Non-blocking flag
                        (struct sockaddr *)&from_addr, &len);
if (recv_len > 0) {
    // Data received, process it
}
```

---

7. HTTP Protocol Basics

HTTP is the standard protocol for IoT devices to interact with cloud REST APIs — devices use GET to fetch configuration, POST to report sensor data, and Range requests to implement OTA firmware chunk downloads. Understanding the request/response format and status codes is the prerequisite for calling Web APIs.

7.1 Request Format

```
GET /index.html HTTP/1.1\r\n          <- Request line: method + URI + version
Host: example.com\r\n                 <- Request headers: key-value pairs
User-Agent: ESP32\r\n
Accept: */*\r\n
\r\n                                  <- blank line = end of headers
                                      <- Request body (empty for GET, content for
POST)
```

7.2 Response Format

```
HTTP/1.1 200 OK\r\n                      <- Status line: version + status code +
reason phrase
Content-Type: text/html\r\n              <- Response headers
Content-Length: 125\r\n
Connection: close\r\n
\r\n                                     <- blank line = end of headers
<html>...response body...</html>          <- Response body
```

7.3 Common Status Codes

Status Code	Meaning	Description
200	OK	Request succeeded
206	Partial Content	Range request returns partial content (OTA firmware header download)
301	Moved Permanently	Permanent redirect
302	Found	Temporary redirect
400	Bad Request	Malformed request
404	Not Found	Resource does not exist
500	Internal Server Error	Internal server error

7.4 Common HTTP Methods

Method	Semantics	Request Body	Typical Use
GET	Fetch a resource	None	Querying data, fetching web pages
POST	Submit data	Yes	Submitting forms, uploading sensor data
PUT	Replace a resource	Yes	Updating configuration
DELETE	Delete a resource	None	Deleting records

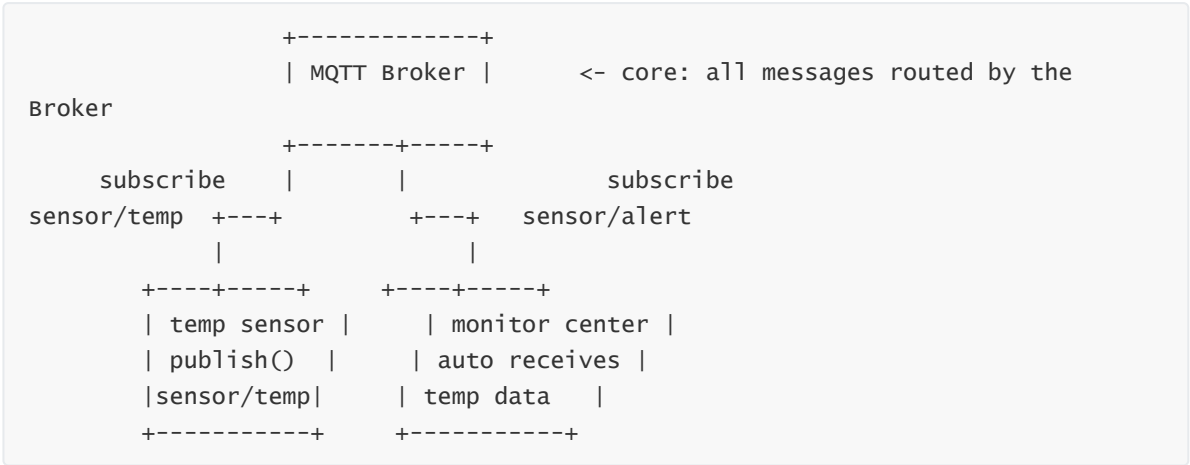
---

8. MQTT Protocol Basics

8.1 Why IoT Uses MQTT Instead of HTTP

Comparison	MQTT	HTTP
Communication mode	Publish/subscribe (Broker relay)	Request/response (direct)
Message push	Native support, subscribe to get pushed	Needs client polling (or WebSocket)
Minimum protocol header	2 bytes	Hundreds of bytes
Connection keep-alive	Keep Alive heartbeat	Stateless (each request is an independent connection)
One-to-many	Naturally supported (many subscribe to the same Topic)	Needs server polling + push logic

8.2 Publish/Subscribe Model



Three roles:

Role	Description	Example
Publisher	Sends messages to a specific Topic	ESP32 reports temperature
Subscriber	Subscribes to a Topic and automatically receives messages	Phone App subscribes to sensor data
Broker	Relays all messages, manages Topic subscription relationships	EMQX, Mosquitto

8.3 Topic Wildcards

Wildcard	Semantics	Example
`+`	Matches **one level**	`home/+/light` -> `home/bedroom/light`
`#`	Matches **multiple levels**	`home/#` -> `home/bedroom/light/status`

8.4 The Three QoS Levels

QoS	Semantics	Mechanism
0	At most once (fire and forget)	No retry, no acknowledgment
1	At least once (may duplicate)	Send -> wait for PUBACK -> resend on timeout
2	Exactly once (no duplicates)	PUBREC -> PUBREL -> PUBCOMP four-step handshake

---

9. ESP-IDF Network API Selection Guide

When developing network applications, ESP-IDF provides APIs **from the bottom to the top**; choose based on your needs:

```
High level (simple, well-encapsulated features)
  |  esp_http_client / esp_http_server    <- HTTP request/server
  |  mqtt_client                          <- MQTT client
  |  esp_ota                              <- OTA upgrade
  |
  |  lwip/sockets (BSD Socket API)        <- raw TCP/UDP communication
  |    socket / bind / listen / accept / connect
  |    send / recv / sendto / recvfrom
  |
Low level (flexible, manual handling of details needed)
  |  esp_wifi                             <- WiFi connect/scan/config
  |  esp_netif                            <- network interface management
  |  esp_event                            <- event-driven framework
```

If you want to...	Use this API
Connect WiFi	`esp_wifi` + `esp_netif`
Custom TCP/UDP communication	`lwip/sockets` (BSD Socket)
Access web pages / REST APIs	`esp_http_client`
Set up a web server	`esp_http_server`
IoT sensor reporting / remote control	`mqtt_client`
WiFi remote firmware upgrade	`esp_ota` + `esp_http_client`
Ping a remote host	`lwip/ping.h` or `esp_ping`

---

10. Common Debugging Methods

Network troubleshooting uses a **layered localization** strategy — confirm level by level from the bottom up: WiFi connection -> IP acquisition -> LAN reachability -> port listening -> application-layer protocol. Skipping levels can easily lead you astray.

10.1 Troubleshooting Steps

```
Problem: ESP32 cannot communicate

1. First check the serial log for "Got IP: xxx.xxx.xxx.xxx"
   |-- Not there -> WiFi not connected -> check SSID/password/band
   `-- There -> WiFi is fine, continue troubleshooting

2. Ping test
   |-- ping ESP32 IP     -> verify whether the IP layer works
   |-- ping router IP    -> verify LAN reachability
   `-- ping 8.8.8.8      -> verify Internet reachability (DNS-independent, pure
IP)

3. Port test
   |-- On the PC, use netstat -an | findstr 8080 to check whether the port is
listening
   `-- Use a network debugging assistant to send data to the ESP32 IP:PORT

4. Firewall check
   `-- The Windows firewall may block non-standard ports such as 8080
```

10.2 Common Debugging Commands

Common network debugging commands for the PC and ESP32 sides — first confirm network reachability from the PC side, then confirm the running status from the ESP32 logs.

```
# PC side (Windows PowerShell)
ipconfig                    # Check local IP
ping 192.168.1.100          # Ping ESP32
netstat -an | findstr 8080  # Check port 8080 status
curl http://192.168.1.100/  # Test HTTP server

# ESP32 side (add in code)
ESP_LOGI(TAG, "Got IP: " IPSTR, IP2STR(&ip));  // Print obtained IP
// Check free memory
ESP_LOGI(TAG, "Free heap: %lu", esp_get_free_heap_size());
```

---

11. Common Network Problems

Symptom	Cause	Solution
`connect()` returns -1	Server not started or wrong IP	Confirm the PC Server is listening and the IP and port are correct
`bind()` fails	Port occupied	Use another port / wait 2 minutes (TIME_WAIT timeout)
`recv()` returns 0	The peer actively closed the connection	Normal disconnect; `break` then `close()`
Data received by `recv()` is "stuck" together	TCP is a byte stream with no message boundaries	Use an application-layer custom frame header + length or a delimiter protocol
`send()` fails repeatedly	Peer closed / network interrupted	The TCP connection is broken; reconnect with `connect()`
Garbled Chinese text when sending	Inconsistent terminal encoding	Check the encoding settings of the serial assistant and network assistant (UTF-8)
PC cannot receive ESP32 data	ESP32 IP changed / firewall blocking	Check the ESP32's actual IP on the serial port; allow the corresponding port in the PC firewall
HTTP client `HTTP request error`	URL unreachable or DNS resolution failed	Make sure the ESP32 can access the Internet; or use an IP instead of a domain name
MQTT `MQTT_EVENT_CONNECTED` not triggered	Wrong Broker address or no Internet on WiFi	`ping broker.emqx.io` to test Internet reachability